Assignment 10: Monte Carlo Methods

PHYS 4202 - Computational Physics (Fall 2026)
Due: Tuesday, November 3, 2026 at 10:00 am
Total points: 100

Purpose

In Assignment 9, you used random events to estimate probabilities and model radioactive decay. Here you will use random sampling to estimate a mathematical constant and evaluate multidimensional integrals. Begin with Buffon's needle, then find the area shared by overlapping circles and the volume shared by intersecting spheres. The emphasis is on:

- translating geometry into random variables and membership tests
- connecting a measured probability to an integral
- validating a numerical method against a known result
- visualizing fluctuations and the spread across independent experiments

Environment

All work must be done in **Python 3.9 or newer**. You may use the Python standard library, NumPy, and matplotlib. Use `numpy.random.default_rng` and allow a user-specified seed. Do not reset the seed inside a trial or repetition loop. Record the seeds and parameters used for the reported results. Vectorized random draws and membership tests are allowed and encouraged. Implement the sampling and estimators yourself. Do not use a library integration routine or replace simulated events with direct binomial draws.

Code organization

Your code directory must contain the following Python files:

- `buffon.py`: Buffon's needle simulations and plots.
- `integration.py`: circle-area and sphere-volume simulations and plots.

Additional helper modules are optional.

Logging and command-line control (10 points)

Use Python's standard-library `logging` module in both `buffon.py` and `integration.py`. Include at least one meaningful statement at each level in **each script**, preferably more where useful. Use `INFO` for execution progress, `DEBUG` for function calls with relevant argument and return values, and `VERBOSE` for detailed per-iteration or per-trial values. Choose the placement and wording yourself. Messages must reflect actual execution and computed values. Define `VERBOSE` yourself as a positive integer below `logging.DEBUG` and register its name with `logging.addLevelName`. You may use

`logger.log` to emit messages at this custom level.

In both scripts, use `argparse` to accept a logging-level option, such as `--log-level`, supporting at least `INFO`, `DEBUG`, and `VERBOSE`. Default to `INFO`. Selecting `DEBUG` should add function-call details. Selecting `VERBOSE` should also include iteration details. Logging must not change the random samples or numerical results.

Document the option and example commands in the report PDF, and verify the level filtering with small runs. Vectorized calculations remain encouraged. Detailed messages may describe individual trials within a batch. You need not log every function or iteration. See the appendix for examples of defining `VERBOSE` and configuring logging with `argparse`. Logging is worth **10 points** within the existing 100-point total.

Part 1: Buffon's Needle (35 points)

Geometry and estimator

A needle of length ℓ is dropped onto a surface covered in parallel lines separated by distance d. Restrict the simulation to $0 < \ell \leq d$. Let x be the distance from the needle's center to the nearest line, and let θ be the acute angle between the needle and the lines. For independent random drops, sample independently and uniformly:

$$x \in [0, d/2] \qquad \theta \in [0, \pi/2]$$

A crossing occurs when

$$x \leq \frac{\ell}{2} \sin\theta$$

For uniformly sampled drops, the geometry predicts that a single needle crosses a line with probability

$$P = \frac{2\ell}{\pi d}$$

This relates the crossing probability to the known needle length ℓ, the line spacing d, and the constant π. The simulation estimates this probability by counting crossings. After N drops, let H be the number that crossed a line. The observed crossing fraction is

$$\widehat{P} = \frac{H}{N}$$

The hat indicates an estimate based on a finite random sample. As more drops are sampled, this fraction should approach the predicted probability P.

To estimate π, solve the probability relation for π and substitute the measured fraction $\widehat{P}$ for P. This gives

$$\widehat{\pi} = \frac{2\ell}{d\widehat{P}} = \frac{2\ell N}{dH} \qquad (H > 0)$$

Thus the calculation proceeds from random needle drops to a crossing count, then to an estimated probability, and finally to an estimate of π. For a running estimate after j drops, use j in place of N and the number of crossings among those first j drops in place of H.

Explain how the crossing condition follows from the needle's projection perpendicular to the lines. Show how averaging over x and θ gives the stated probability. Using NumPy's value of π to specify angular sampling and reference lines is allowed. The measured crossing count must determine $\widehat{\pi}$.

A single experiment

Use $\ell = d = 1$ in consistent chosen length units and $N = 100{,}000$ drops.

- Plot the running estimate $\widehat{\pi}(j)$ against the number of drops j, using a logarithmic horizontal axis and a reference line at π. You may record selected trial intervals instead of storing every intermediate estimate.
- Report ℓ, d, N, H, $\widehat{P}$, $\widehat{\pi}$, and the relative error $(\widehat{\pi} - \pi)/\pi$. Explain the early fluctuations and why convergence need not be monotonic.

The probability estimate is defined even when $H = 0$, but $\widehat{\pi}$ is not. Mark such estimates as undefined and omit them from plots. Do not replace a zero count with one or restart an experiment to obtain a crossing.

Repeated experiments: visualizing the spread

Run $M = 30$ independent experiments, each with $\ell = d = 1$ and $N = 100{,}000$ drops. Use fresh random draws for each experiment and track its running estimate $\widehat{\pi}(j)$ throughout the run.

- Make a **separate figure from the single-experiment plot**. Overlay the running estimates from all 30 experiments over $1 \le j \le N$, using a logarithmic horizontal axis and a reference line at π. Use thin lines and transparency so the spread among experiments remains visible. You may record selected trial counts spanning the full run rather than every intermediate estimate. Omit undefined estimates before an experiment's first crossing.
- Describe how the spread changes as more drops are sampled. Use the figure to discuss the variability in an estimate obtained from a finite random sample.

Part 2: Monte Carlo Integration (40 points)

Area and volume as integrals

Let Ω be a region contained in a bounding region B. In two dimensions, B is a rectangle. In three dimensions, it is a box. Define the indicator function

$$\chi_\Omega(\mathbf{r}) = \begin{cases} 1, & \mathbf{r} \text{ is inside } \Omega, \\ 0, & \mathbf{r} \text{ is outside } \Omega. \end{cases}$$

The area or volume of the region is

$$A_\Omega = \int_B \chi_\Omega(\mathbf{r})\, d^2r, \qquad V_\Omega = \int_B \chi_\Omega(\mathbf{r})\, d^3r$$

Sample N independent points uniformly throughout B. If H points lie in Ω, then the Monte Carlo estimates are

$$\widehat{A} = A_B \frac{H}{N}, \qquad \widehat{V} = V_B \frac{H}{N}$$

Here A_B is the area of the sampling rectangle and V_B is the volume of the sampling box. Explain why uniform sampling converts each integral into the size of the bounding region times a fraction.

Two offset circles

Begin in two dimensions with two filled circles of radius R, centered at $(-R/2, 0)$ and $(R/2, 0)$. Their overlap satisfies **both** inequalities

$$(x + R/2)^2 + y^2 \leq R^2, \qquad (x - R/2)^2 + y^2 \leq R^2$$

Use $R = 1$ and sample $N = 10{,}000$ points uniformly in the rectangle

$$B = [-3R/2, 3R/2] \times [-R, R], \qquad A_B = 6R^2$$

This rectangle contains both complete circles, including their overlap. The overlap integral and its evaluated reference area are provided:

$$A_{\text{overlap}} = 4 \int_0^{R/2} \sqrt{R^2 - (x + R/2)^2}\, dx = \left(\frac{2\pi}{3} - \frac{\sqrt{3}}{2} \right) R^2$$

Use this reference value to check your Monte Carlo result.

- Classify every sampled point into one of four categories: neither circle, the left circle only, the right circle only, or both circles. Count the points in each category. Only points in both circles contribute to the overlap area estimate.
- Report the four category counts, the overlap fraction H/N, the estimated area $\widehat{A}/R^2$, and its relative error compared with the reference area. Check that the four category counts sum to N and that the counts exclusively within each circle are close to equal.
- Make a scatter plot showing **all** N **sampled points**, with four distinct colors for the four categories. Include the two circle boundaries and mark their centers. Use equal axis scales (ensure the circles remain circular) and provide a legend. Use small markers and transparency so the point distribution remains readable.

Two offset spheres

Consider two solid spheres of equal radius R, centered at

$$\mathbf{c}_1 = (-R/2, 0, 0), \qquad \mathbf{c}_2 = (R/2, 0, 0).$$

A point belongs to a sphere if its distance from that sphere's center is at most R. The volume shared by the two spheres satisfies both inequalities

$$(x + R/2)^2 + y^2 + z^2 \leq R^2 \qquad (x - R/2)^2 + y^2 + z^2 \leq R^2$$

The overlap lies between $x = -R/2$ and $x = R/2$. Use the bounding box

$$B = [-R/2, R/2] \times [-R, R] \times [-R, R] \qquad V_B = 4R^3$$

Sample each coordinate independently and uniformly over its own box interval. Use $R = 1$ in consistent chosen length units for the required studies. The exact shared volume is provided as a reference:

$$V_2 = \frac{5\pi}{12} R^3$$

- Implement the two-sphere membership test and use Monte Carlo integration to estimate the shared volume. Compare your estimate with the reference value above.
- With $N = 100{,}000$, report H, $\widehat{p}$, $\widehat{V}_2/R^3$, and the relative error compared with V_2.
- Plot the running estimate $\widehat{V}_2(j)/R^3$ versus j with a logarithmic horizontal axis and a reference line at the exact value. Discuss fluctuations and agreement at large sample counts.

Three offset spheres

Add a third sphere of radius R, centered at $\mathbf{c}_3 = (0, R/2, 0)$. The common volume must satisfy both previous inequalities **and**

$$x^2 + (y - R/2)^2 + z^2 \leq R^2.$$

All three centers are offset from the origin. Use the same bounding box and the same sampled points to estimate both V_2 and V_3. The three-sphere points are a subset of the two-sphere points, so the two estimates from a shared sample are correlated.

- With $N = 100{,}000$, report the three-sphere hit count, acceptance fraction, and $\widehat{V}_3/R^3$. An exact expression for V_3 is not required. Support your result with the two-sphere validation and the geometric checks below.
- Explain why the third sphere reduces the volume and why an intersection requires logical AND between the tests. Contrast this with the membership rule for a union of the spheres within the box.

Geometric checks

Check that every three-sphere hit is also a two-sphere hit, that $0 \leq \widehat{V}_3 \leq \widehat{V}_2 \leq 4R^3$, and that known inside and outside points are classified correctly. Include a point inside the first two spheres but outside the third. Summarize your checks in the report.

Figures, short report, and submission (15 points)

Prepare a concise report in LaTeX and submit the compiled PDF to Canvas. Include your name, a descriptive title, and your GitHub repository URL. Present a connected analysis supported by the requested figures, numerical results, comparisons, and checks. Include a brief testing or implementation note.

Include every required figure in the report. Label axes, identify curves, and provide concise captions. Use logarithmic trial-count axes for running estimates and keep the two Buffon's needle figures separate. Put all documentation in the report, including brief run instructions, dependencies, parameters, seeds, and example commands for selecting the logging level.

Include an **AI-use statement**. If you used AI, name the tools, describe their contributions, and explain how you checked or revised the work. If you did not use AI, say so.

Commit the Python code and any supporting modules to your GitHub repository. Submit the report PDF to Canvas. Do not submit plot files separately or commit them to the repository. Keep generated figures out of version control using `.gitignore`. The figures are assessed within the report.

Grading (100 points)

Part 1: Buffon's needle (35 points)

Category	What I look for	Pts
Simulation and geometry	Sampling, crossing test, and probability derivation	10
Single experiment	Running estimate, results, and interpretation	10
Repeated experiments	Independent runs, overlaid estimates, and spread	15
Total		35

Part 2: Monte Carlo integration (40 points)

Category	What I look for	Pts
Circle overlap	Area, four-color plot, and reference comparison	15
Two-sphere integration	Sampling, volume, running plot, and comparison	15
Three-sphere geometry	Intersection, volume, and checks	10
Total		40

Logging and command-line control (10 points)

Category	What I look for	Pts
Logging statements	INFO, DEBUG, and VERBOSE in both scripts	5
Configuration and checks	Custom level, ArgParse option, and filtering	5
Total		10

Report and documentation (15 points)

Category	What I look for	Pts
Presentation	Clear figures, labels, captions, and organization	5
Discussion	Analysis supported by results and checks	5
Documentation	Run instructions, repo URL, testing note, and AI statement	5
Total		15

Appendix: Custom VERBOSE Level and ArgParse

These examples show how to configure logging. Choose the placement and content of messages throughout your own simulations.

Defining VERBOSE

Python does not provide a built-in `VERBOSE` level. Define a numeric level below `DEBUG`, which has value 10, and register its name:

```
import logging

VERBOSE = 5
logging.addLevelName(VERBOSE, "VERBOSE")
logger = logging.getLogger(__name__)
```

Use `logger.log(VERBOSE, ...)` when emitting a message at this level.

Selecting the level with argparse

The following continues the setup above. Add argument parsing and configure logging before the simulation begins:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument(
    "--log-level",
    type=str.upper,
    choices=["INFO", "DEBUG", "VERBOSE"],
    default="INFO",
    help="Logging detail (default: INFO)",
)
args = parser.parse_args()

levels = {
    "INFO": logging.INFO,
    "DEBUG": logging.DEBUG,
    "VERBOSE": VERBOSE,
}
logging.basicConfig(
    level=levels[args.log_level],
    format="%(levelname)s: %(message)s",
)

logger.info("Starting simulation")
logger.debug("Selected log level: %s", args.log_level)
logger.log(VERBOSE, "Detailed logging enabled")
```

For example, run `python buffon.py` for the default `INFO` level, `python buffon.py --log-level DEBUG` for additional debugging messages, or `python buffon.py --log-level VERBOSE` for all three message types. The same option should be available in `integration.py`.